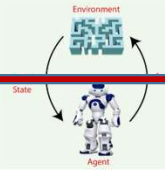
Reinforcement Learning

Dr. D. John Pradeep

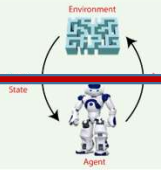
Associate Professor

VIT-AP University



Syllabus

Module No. 1	**REINFORCEMENT LEARNING – Fundamentals**	**8 Hours**
Introduction to Reinforcement Learning–Reinforcement Learning framework and application–Introduction to Immediate Reinforcement Learning –Bandit Algorithm Introduction-Bandit Algorithm, UCB, PAC, Bandit Optimality–Value function based method-Policy Gradient		
Module No. 2	**MDP**	**8 Hours**
Full RL introduction – Return, values Function – Introduction to MDP model- Bellman Equation –Optimization of Bellman Equation – Cauchy sequence and Green Equation- Banach Fixed Point Theorem- Convergence Proof.		
Module No. 3	**Deep reinforcement learning**	**8 Hours**
Dynamic Programming – Monte Carlo – Components –Control in Monte Carlo–LPI Convergence, Value Iteration, Policy Iteration– Q-Learning- Q-learning with deep networks, Double Q learning, replay memory, Deep NN Architectures for RL		
Module No. 4	**Eligibility Traces and Function Approximation**	**7 Hours**
Eligibility Traces – Backword view of eligibility traces- Eligibility Trace Control – Function Approximation – Function Approximation and Eligibility Traces- Least Square Method		
Module No. 5	**Fitted Q and DQN**	**6 Hours**
DQN and Fitted Q iterations – Policy Gradient Approach – Policy Gradient approach with function approximation		
Module No. 6	**Hierarchical Reinforcement Learning**	**8 Hours**
Hierarchical Reinforcement Learning –Types of Optimality– Semi MDP Model – Options – Learning with Options– Hierarchical Abstract Machines- A partially observable Markov decision process.		



Evaluation - Theory

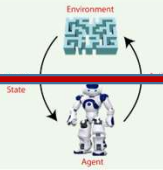
Component	Max Marks	Weightage
CAT - 1	50	15
CAT - 2	50	15
FAT	100	40
Digital Assessment - 1	10	5
Digital Assessment - 2	10	10
Quiz - 1	10	5
Quiz - 2	10	10

Evaluation - Project

Component	Max Marks	Weightage
Review - 1	100	25
Review – 2	100	50
Report	25	25

Table of contents

- Introduction
- Supervised vs Unsupervised learning
- Reinforcement Learning
- RL – Exploring vs Exploiting
- Elements of RL



Course Outcomes

Course Outcomes	Course Outcome Statement	PO's / PEO's
CO1	Knowledge of basic and advanced reinforcement learning techniques	PO1, PO2, PO3, PO4, PO5, PO6, PO8, PO11
CO2	Identification of suitable learning tasks to which these learning techniques can be applied	PO1, PO2, PO4, PO6, PO7, PO11
CO3	To develop basic mathematical model of various real time data and learned a model	PO1, PO2, PO3, PO4, PO9, PO10, PO11, PO12
CO4	Evaluate prediction based on learning and real result and obtain the difference in terms of error percentage	PO2, PO3, PO4, PO5, PO6, PO11
CO5	Propose the use of certain technologies by implementing them in the python programming language to solve the given problem	PO1, PO2, PO3, PO4, PO2, PO11
CO6	Choose an engineering approach to solving problems, starting from the acquired knowledge of programming and knowledge	PO3, PO4, PO5, PO6, PO9, PO11, PO12

Program Objectives and Educational Objectives

Engineering Graduates will be able to:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

Program Objectives and Educational Objectives

Engineering Graduates will be able to:

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

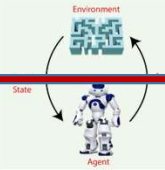
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



Program Objectives and Educational Objectives

Program Educational Objectives (PEOs)

PEO1: Core Competency:

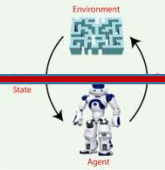
To impart fundamentals of mathematics, science, and electronics engineering for developing critical thinking, problem-solving ability and leadership skills.

PEO2: Professional and responsible competency:

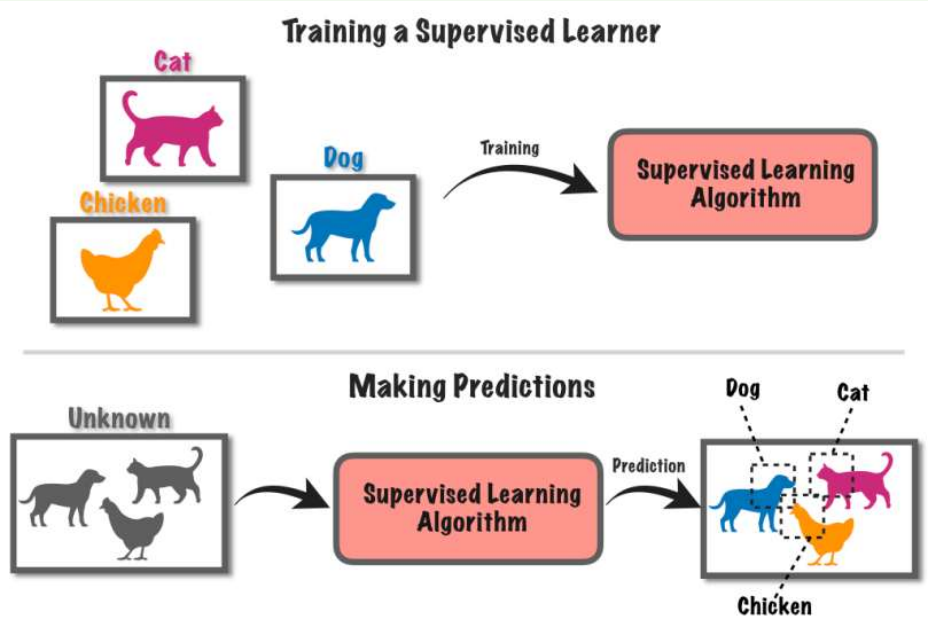
To provide methodologies of electronic system design, analysis, optimization and development considering social and industrial needs.

PEO3: Life-long Learning:

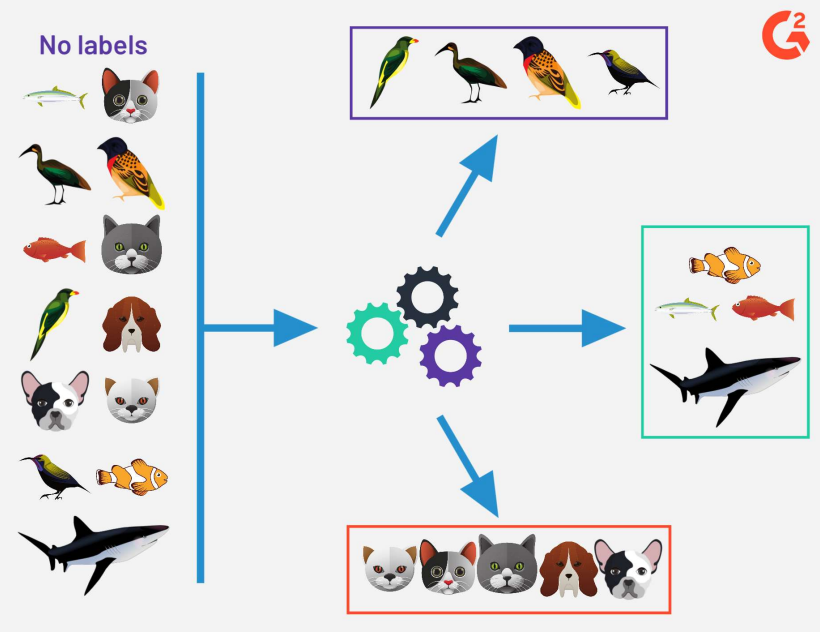
To encourage updating and applying the knowledge base in administrative skills through higher education, professional training and courses ensuring a better future.



Introduction



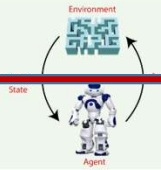
Supervised Learning



Unsupervised Learning

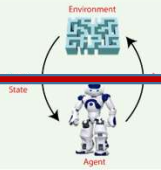
Supervised vs Unsupervised learning

Supervised Learning	**Unsupervised Learning**
Uses labelled data for training purpose	Uses unlabeled data for training
Model requires feedback to train (to check if the prediction made is correct or not)	No feedback mechanism
Output is predicted by the model	Hidden lying patterns in the data is learnt
Model is trained such that it can predict the output when new data is presented	Main goal is to find the hidden patterns in the input dataset.
Algorithms used: Linear regression, logistic regression, SVMs, Multiclass classification, Decision tree, Bayesian logic etc.,	Algorithms used: clustering, knn, apriori algorithm etc.,
Not AI	True AI

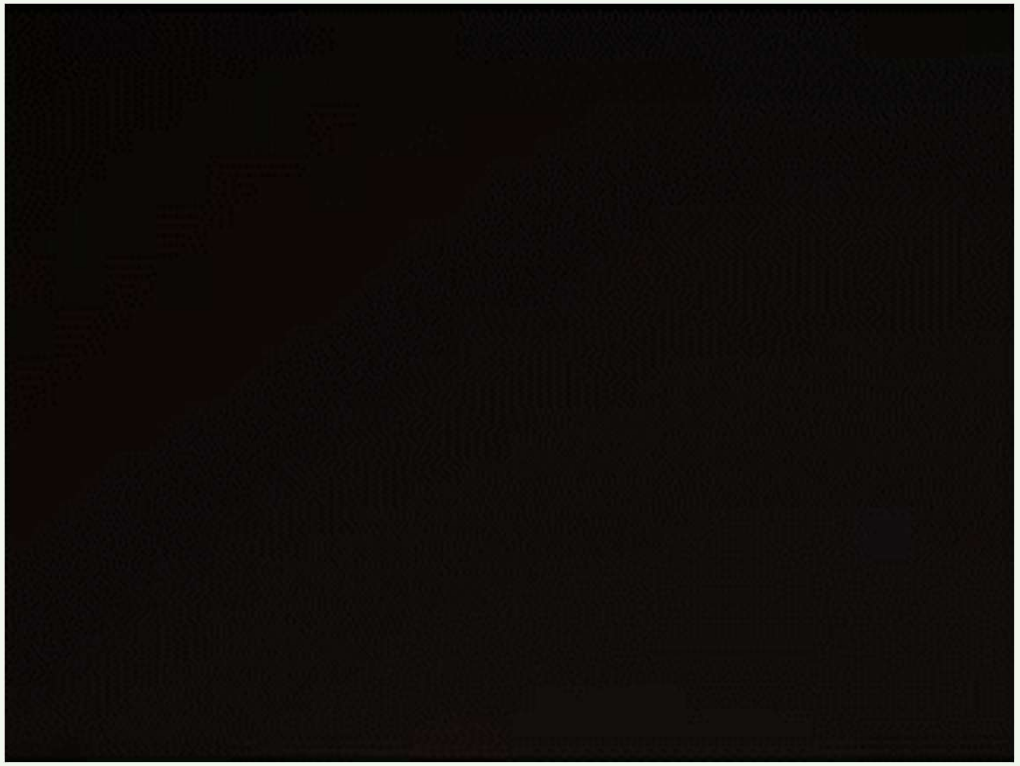


Reinforcement Learning

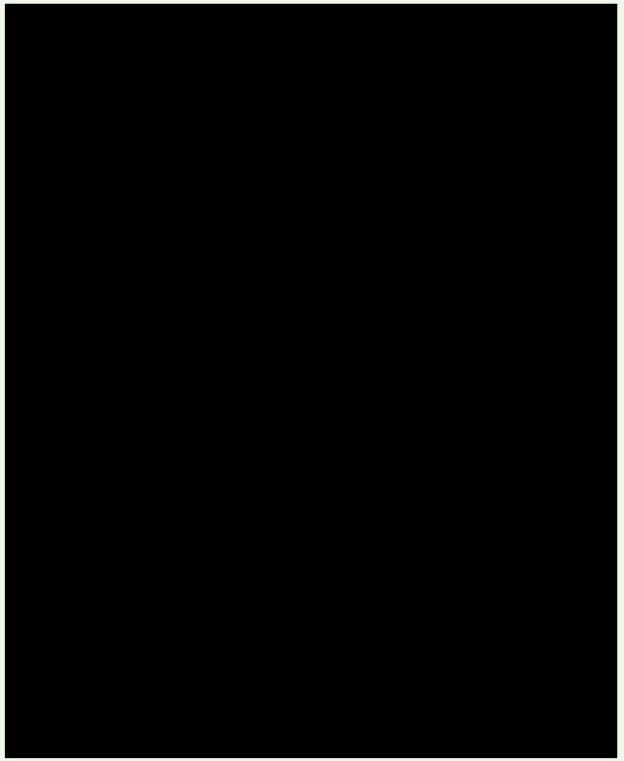
- Uses feedback to train the model – Supervised learning?
- Doesn't use labelled data – Unsupervised learning??
- It is hybrid machine learning process
- Mimics human learning pattern
- Trial and error based learning

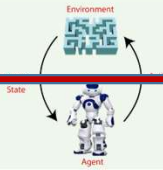


RL Application – Inverted Helicopter flight

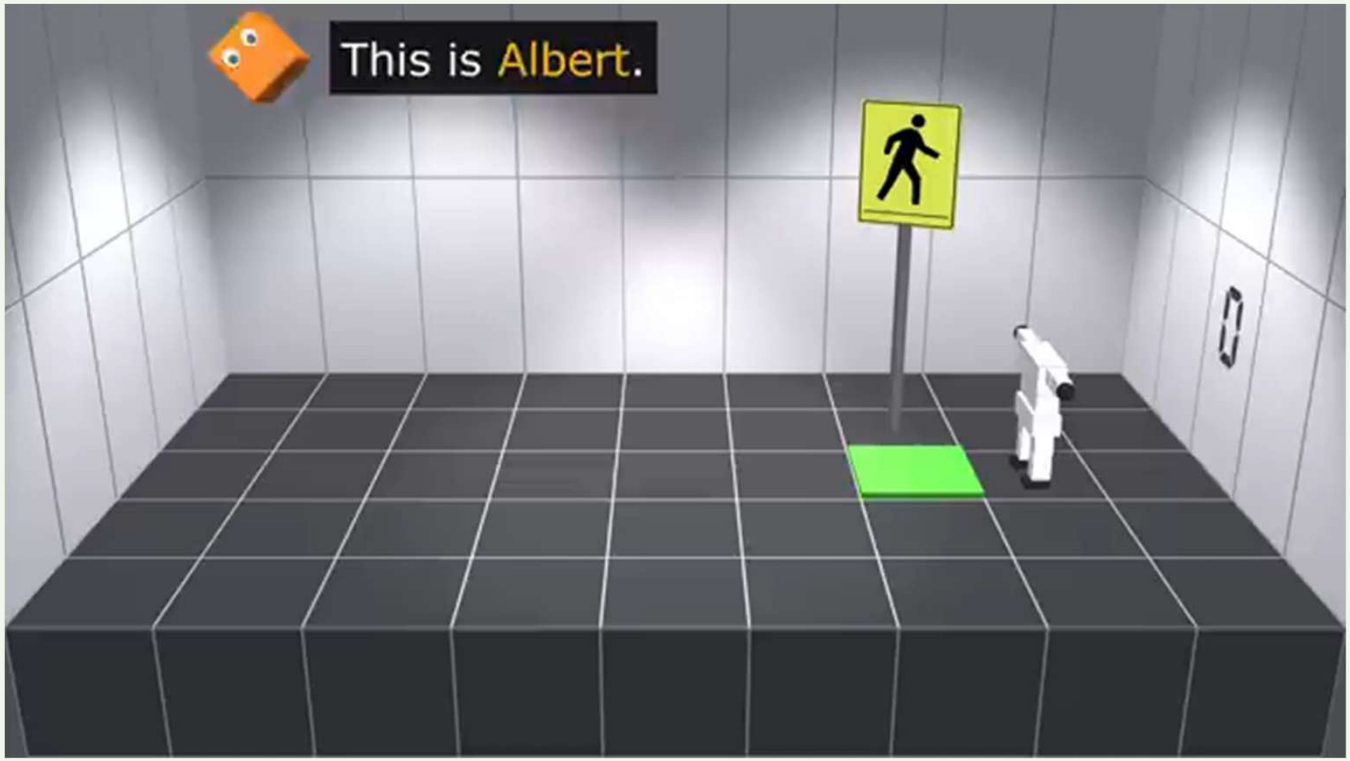


RL Application – Atari game

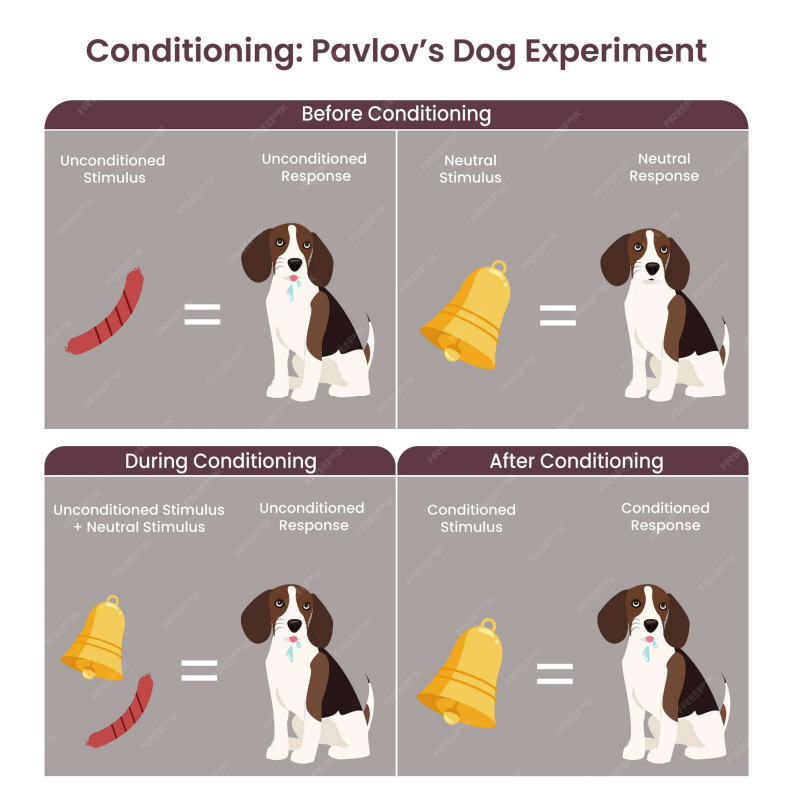




RL Application – Learning to walk



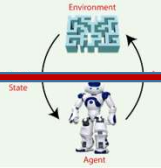
Reinforcement Learning



- Ivan Petrovich Pavlov (1849 – 1936) is a Russian physiologist discovered classical conditioning (Pavlov Conditioning) by accident.
- Classical conditioning involves learning to associate an unconditioned stimulus (food) that already brings about a particular response (salivating) with a new conditioned stimulus (ringing of bell), so that the new stimulus brings about the same response.

Reinforcement Learning

- Learning from interaction – idea of RL
- RL is goal directed learning from interaction
- RL is all about learning what to do (action) in a particular situation (state)
- Mapping actions to states by maximizing a numerical reward.
- RL problem is formalized as a Markov Decision Process (MDP)
- MDPs are meant to frame the problem of learning from interaction to achieve a goal



Reinforcement Learning

MDP terms

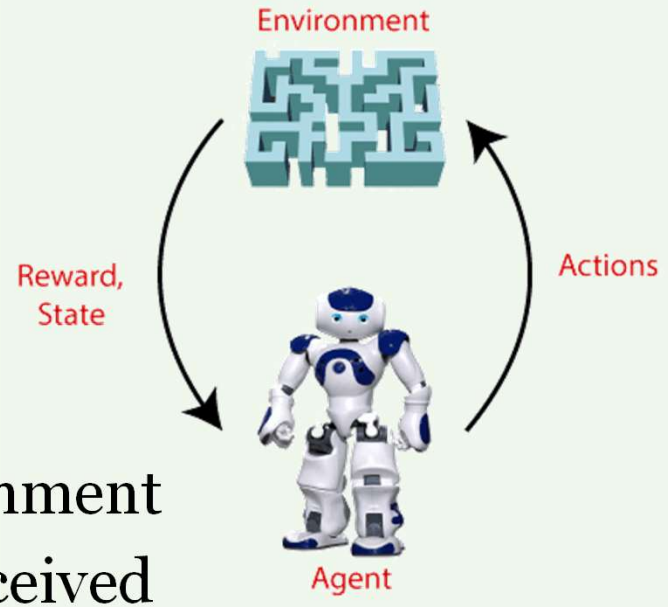
Agent – Learner/Decision maker

Environment – The thing the agent interacts

State – The situation of the environment

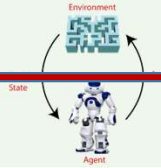
Action – The steps the agent performs on environment. They alter the state of the environment

Reward – The numerical acknowledgment received by the agent for the action performed, from the environment.



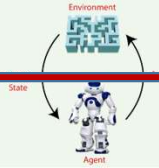
Reinforcement Learning

- RL – take actions to maximize the numerical reward signal
- Learner is not told which action to be taken a given situation but must discover which actions yield most reward by trying them (explore)
- In most interesting cases, actions may not only affect the immediate rewards but also the next situation and, through that, all subsequent rewards.
- Trial and error search and delayed rewards are the two most important distinguishing features of RL



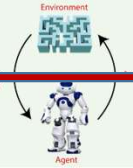
RL – exploration Vs exploitation

- RL agent should discover which actions yield most reward (explore)
- RL agent must also prefer actions taken in the past which provided good reward (exploit)
- The RL agent should **exploit** to obtain better reward but also should **explore** which action gives the best reward.
- The dilemma is that exploration and exploitation cannot be exclusively pursued without failing at the task.
- Which means that if one task is continuously pursued, it leads to failure



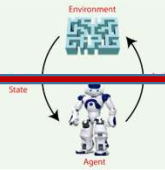
Elements of RL

- Apart from agent, environment, action, state, reward, there are other main elements of RL
- Policy – mapping states to actions (which action to be performed in a specific state)
- Value function – total amount of reward the agent is expected to accumulate over the future starting from that state.
- Model of the environment – It mimics the behavior of the environment – how the environment behaves



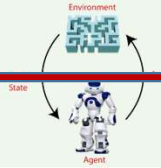
Policy

- Is a rule book which RL agent has learnt how to behave at a given time
- It is a mapping from perceived state of environment to actions to be taken when in those states
- Policy is the core of RL agent and is alone sufficient to determine the behavior of the agent



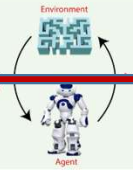
Reward

- Defines the goal of RL problem
- At each time step, the environment sends RL agent a single number called reward
- The agent's sole objective is to maximize the total reward it receives over a long run
- Reward signal defines what are good and bad events for the agent
- They are the immediate and defining features of the problems faced by the agent
- Reward signal is the primary basis for altering the policy
- If an action selected by the policy is followed by low reward, then policy may be changed to select a different action in that situation in future



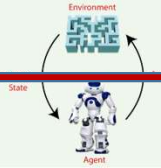
Value function

- Reward indicates what is good in immediate sense
- Value function specifies what is good in long run
- Value is assigned to state where as reward is a feedback from the environment
- Value of a state is the total amount of reward an agent can expect to accumulate over the future starting from that state



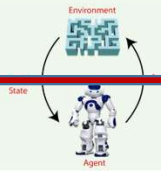
Model of environment

- Model mimics the behavior of the environment
- Given state and action the model may predict next state and reward associated.
- Models are used for planning purpose – actions for future
- If models are specified, we call the learning as model based learning
- If model of the environment is not specified, the behavior of the environment should be understood/learnt by trial and error – model free method



Temporal difference update

- $V(S_t) \leftarrow V(S_t) + \alpha\big(V(S_{t+1}) - V(S_t)\big)$
- Temporal difference learning method - the changes (updates) are based on the difference between estimates at two successive times



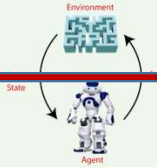
Bandit Problem

- Bandit refers to a slot machine. The figure shown is one armed bandit.
- The term Multi-Armed Bandit (abbreviated as MAB) is a spoof name that stands for “Many One-Armed Bandits” and the term One-Armed Bandit refers to playing a slot-machine in a casino (that has a single lever to be pulled, that presumably addicts us and eventually takes away all our money, hence the term “bandit”).



Goal

- Select the arm (action) which gives you maximum payback
- Find $q^*(a^*)$



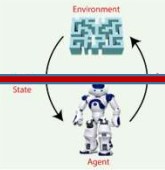
Bandit Problem – Formal definition

A multi-armed bandit (MAB) comprise of

- A finite set of Actions A (known as the arms)
- Each action $a \in A$ is associated with a probability distribution over $\mathbb{R}$ unknown to the agent denoted as R^a, defined as:

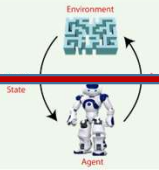
  $R^a(r) = \mathrm{P}(r|a)$ for all $r \in \mathbb{R}$
- A time-indexed sequence of AI agent-selected actions $A_t \in A$ for time steps $t = 1, 2, 3\ldots\ldots$ and a time-indexed sequence of environment-generated random variables $R_t \in \mathbb{R}$ for time steps $t = 1, 2, 3\ldots\ldots$ with R_t randomly drawn from the probability distribution $\mathrm{R}^a(t)$
- The goal of the agent is to maximize the expected sum of cumulative rewards over a certain number of time steps t

$$E\left[\sum_{t=1}^{T} R_t\right]$$



Notions for solving Bandit Problem

- Asymptotic correctness
- Regret Optimality
- PAC optimality/complexity
- Value function based methods
- Softmax
- UCB – Upper Confidence Bound



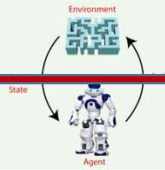
Asymptotic correctness

- As t → infinity,

Give the guarantee that the arm with highest pay-off is selected.

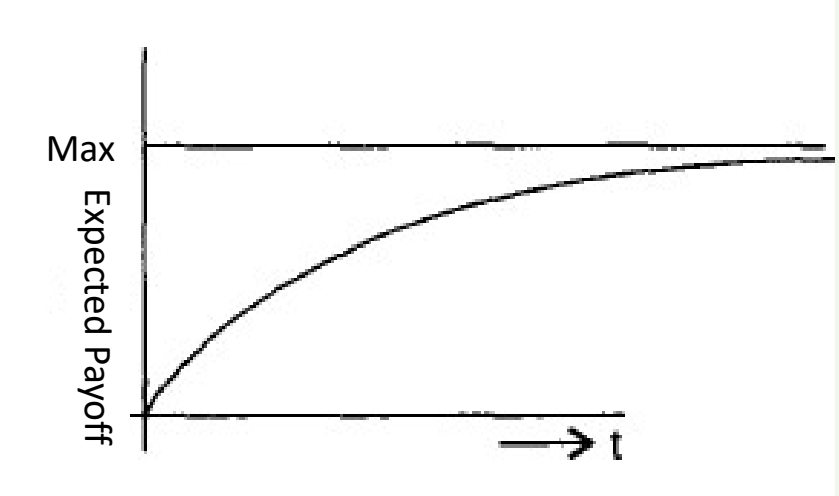
The algorithms used ensure that the $q^*(a)$ computed ultimately converge to $q^*(a^*)$

Here convergence is of high priority



Regret Optimality

- This type of solution is concerned about the “regret” parameter which is - the loss incurred in computing expected payoff because of exploration
- These solutions exploit for steeper characteristics thereby reducing the regret
- No regret can diminish faster than log(T)



Regret

- The loss that we incur due to time/rounds spent due to the learning is called regret.

We define the *Action Value* $Q(a)$ as the (unknown) mean reward of action a, i.e.,

$$Q(a) = \mathbb{E}[r|a]$$

We define the *Optimal Value* V^* and *Optimal Action* a^* (noting that there could be multiple optimal actions) as:

$$V^* = \max_{a \in \mathcal{A}} Q(a) = Q(a^*)$$

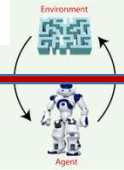
We define *Regret* l_t as the opportunity loss at a single time step t, as follows:

$$l_t = \mathbb{E}[V^* - Q(A_t)]$$

We define the *Total Regret* L_T as the total opportunity loss, as follows:

$$L_T = \sum_{t=1}^{T} l_t = \sum_{t=1}^{T} \mathbb{E}[V^* - Q(A_t)]$$

Maximizing the *Expected Cumulative Rewards* is the same as Minimizing *Total Regret*.



PAC optimality/complexity

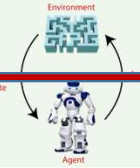
- Probably Approximately Correct
- It is highly probable that the arm "a" returned, gives $q^*(a)$ which is approximately close to $q^*(a^*)$

(ε, δ) – PAC

$$P(q^*(a) > q^*(a^*) - \epsilon) \geq (1 - \delta)$$

Given any number of arms, the optimality is guaranteed if the number of samples drawn are large (infinite)

- PAC Optimality comes into picture when there is sample complexity.
- Given (ε, δ), what is the smallest number of samples that needs to be drawn such that (ε, δ) – PAC is guaranteed



PAC optimality/complexity

Sample complexity (number of iterations)

For a finite MDP with$|S|$ states$|A|$ actions, discount factor γ, the number of non-ε-optimal steps is bounded by:

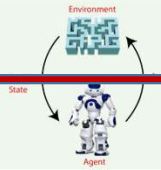
$$N = O\left(\frac{|S||A|}{|\varepsilon^2(1-\gamma)^3|}\log\frac{|S||A|}{\delta}\right)$$

N is the PAC bound on iterations

Interpretation (important)

- ε → accuracy (smaller ε ⇒ more iterations)
- δ → confidence (smaller δ ⇒ more iterations)
- |S||A| → problem size
- (1−γ) → planning horizon

(as γ → 1, required iterations increase sharply)



1. What is an ε-optimal policy?

A policy π is **ε-optimal** if its value is *close* to optimal:

$$V^{\pi}(s) \geq V^{*}(s) - \varepsilon \quad \text{for all states } s$$

Meaning:

- The policy may be slightly worse than optimal
- But the loss is **at most ε**

2. What is a non-ε-optimal step?

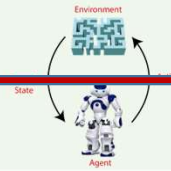
At time step t:

$$V^{\pi_t}(s_t) < V^{*}(s_t) - \varepsilon$$

That step is called a **non-ε-optimal step**.

👉 In simple words:

> **A step where the agent's current policy is more than ε worse than the optimal policy.**



Value function based methods

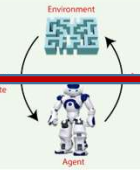
- In these methods, q* is unknown but we can form an estimate of q* which is Q_t
- When action a is taken,

$$Q_t(a) = \frac{\sum_{i=1}^{t} 1_{(a_i=a)} R_i}{\sum_{i=1}^{t} 1_{(a_i=a)}}$$

Greedy: Select action such that

$$arg\max_a Q_t(a) \text{ (exploitation)}$$

Being exploitative will always lead to non exploration of best states

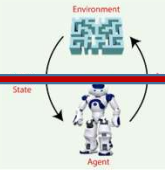


Value function based methods

- ε-greedy: exploration policy
- With probability 1- ε, be greedy with probability ε, we explore by selecting a set of actions uniformly from A – action space

Limitations:

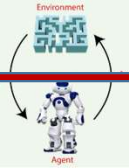
- It doesn't address the problem of unwanted states
- Even if there is a minute difference between the actions, the one which has minutest large value will be selected greedily.



Value function based methods

Summary

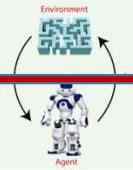
- With probability (1- ε) choose action $arg\max_{a} Q_t(a)$
- With probability ε choose action uniformly from A



Softmax

- Greedy action gets bulk of probability (exploitation) and other actions are given equal probability (exploration) of getting selected
- Exploration is made cleverer in softmax by assigning 0 probability to the states which are unwanted
- The probability of selecting an action is proportional to the current estimate of Q_t

$$P(a_t = a) = \frac{Q_t(a)}{\sum_{b=1}^{n} Q_t(b)} \text{ (intuitive)}$$



Softmax

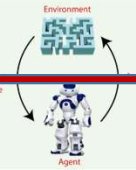
- The probability may be negative because of the rewarding process (different signs/weightages). To avoid this, exponents of the estimates of Q_t are considered

$$P(a_t = a) = \frac{e^{Q_t(a)/\beta}}{\sum_{b=1}^{n} e^{Q_t(b)/\beta}}$$

β – Temperature parameter

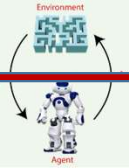
If β is very high, all actions will have same probability – random

If β is very low, exploitation - greedy



Softmax

- As this method explores with probability ε, there is a possibility that the solution oscillates around q*
- So, to avoid this, we need to cool ε (cooling of ε proportional to 1/t solves the problem)
- But cooling the ε will take longer period for convergence – need a balance

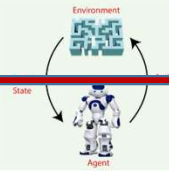


Softmax

How can the Softmax algorithm be applied to solve a multi-armed bandit problem with 3 arms over 8 iterations, using the provided data? Experiment with temperature parameters (β) of 0.2 and 0.6 to demonstrate their impact on the balance between exploration and exploitation. **(10 M)**

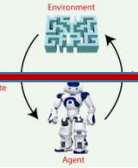
Iteration	1	2	3	4	5	6	7	8
Action	B	C	A	B	C	A	B	C
Reward	5	2	3	1	5	4	3	2

Table 1

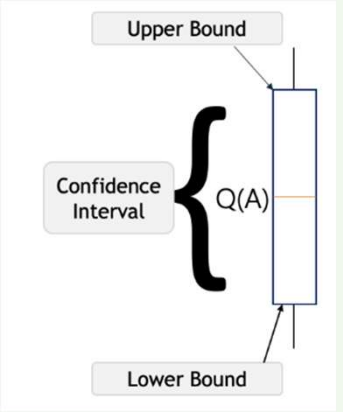


UCB – Upper Confidence Bound

- Assume K – arms. Each arm is associated with some arbitrary distribution function. When an arm is pulled, we receive a reward from the associated distribution.
- Upper-Confidence Bound action selection uses uncertainty in the action-value estimates for balancing exploration and exploitation.
- Since there is inherent uncertainty in the accuracy of the action-value estimates when we use a sampled set of rewards thus UCB uses uncertainty in the estimates to drive exploration.



UCB – Upper Confidence Bound



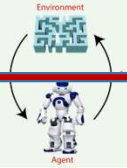
Algorithm:

1. Play each arm atleast once
2. Do the following in loop

Play arm A_t that maximizes $Q_t(a) + \sqrt{\frac{2\ln(n)}{N_t(a)}}$ *i.e.*, $A_t = \arg\max_a (Q_t(a) + c\sqrt{\frac{2\ln(n)}{N_t(a)}})$

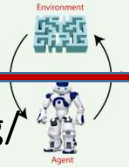
- $Q_t(a)$ here represents the current estimate for action a at time t. We select the action that has the highest estimated action-value plus the upper-confidence bound exploration term.
- The lower bracket is called the lower bound, and the upper bracket is the upper bound.
- The region between the brackets is the confidence interval which represents the uncertainty in the estimates.

Ref: Upper Confidence Bound Algorithm in Reinforcement Learning - GeeksforGeeks



UCB – Upper Confidence Bound

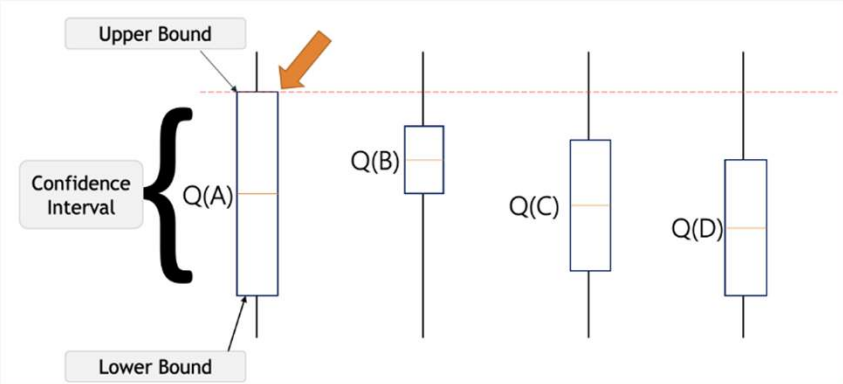
- If the confidence region is very small, then we become very certain that the actual value of action *A* is near our estimated value.
- On the other hand, if the region is large, then we become uncertain that the value of action *A* is near our estimated value.

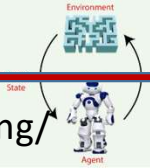


Ref: https://www.geeksforgeeks.org/upper-confidence-bound-algorithm-in-reinforcement-learning/

UCB – Upper Confidence Bound

- let's say we have these four actions with associated uncertainties in the picture below
- The agent has no idea which is the best action. So according to the UCB algorithm, it will optimistically pick the action that has the highest upper bound i.e. *A*.

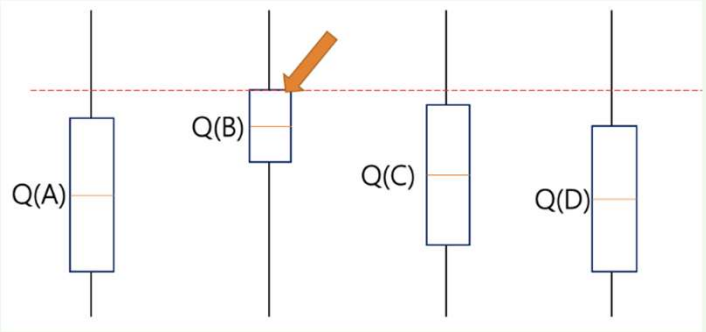




Ref: https://www.geeksforgeeks.org/upper-confidence-bound-algorithm-in-reinforcement-learning/

UCB – Upper Confidence Bound

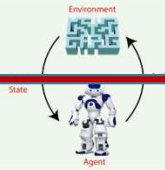
- By doing this either it will have the highest value and get the highest reward, or by taking that we will get to learn about an action we know least about.
- Let's assume that after selecting the action *A* we end up in a state depicted in the picture below. This time UCB will select the action *B* since *Q(B)* has the highest upper-confidence bound because it's action-value estimate is the highest, even though the confidence interval is small.
- UCB explores more to systematically reduce uncertainty but its exploration reduces over time.



Ref: https://www.geeksforgeeks.org/upper-confidence-bound-algorithm-in-reinforcement-learning/

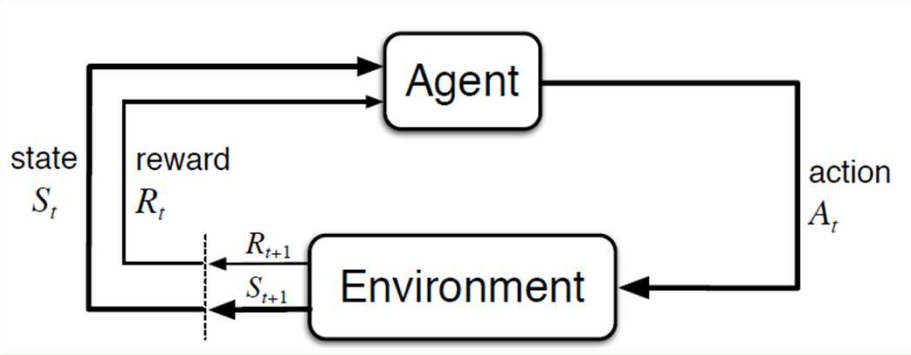
Notions for solving Bandit Problem

- Asymptotic correctness – problem of correctness
- Regret Optimality – problem of convergence
- PAC optimality/complexity – problem of sample complexity



Full RL

- Here $S_t \in S, At \in A(s), R_t \in R$
- The trajectory followed is $S_0\, A_0\, R_1\, S_1\, A_1\, R_2\, S_2\, A_2$.....
- In finite MDP, **S**, **A**, **R** all are finite meaning they have finite number of elements
- R_t and S_t are random variables and have a well defined probability distribution.



Reward and policy

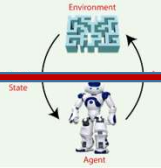
- Reward is a scalar function
- Agent has no prior knowledge on how to get rewards – out of direct control of the agent
- Bounded and frequent

Policy

$$\pi_t(a|s) \stackrel{\text{def}}{=} P[A_t = a|S_t = s]$$

Policy defines the probability that an action a is selected in state s at time t

Normally we deal with stationary policies



Returns (deals with long term)

- General notation

$G_t = R_{t+1} + R_{t+2} + R_{t+3} \ldots\ldots\ldots + R_T$ (Sum of all the rewards we receive in future)

T – end of episode (eg. End of chess game) – random variable

Goal: To maximize returns

- Notion 2

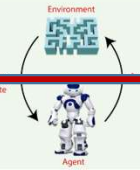
$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \ldots\ldots$ $(0 \leq \gamma < 1)$ γ – discount factor

The reward received now is more valuable than the reward we may get in future

Notion 3

$G_t = \frac{1}{N}[R_{t+1} + R_{t+2} + R_{t+3} \cdots + R_{t+N}]$ - for finite N

$G_t = \lim_{N \to \infty} \frac{1}{N}[R_{t+1} + R_{t+2} + R_{t+3} \cdots + R_{t+N}]$ for infinite N



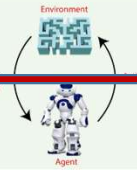
Full RL

- For particular values of random variables, $s' \in S$ and $r \in R$, there is a probability of those values occurring at time t, given particular values of the preceding state and action:

$$p(s', r \mid s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}$$

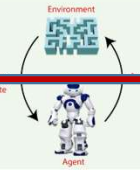
Satisfies

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$



Full RL

- In a Markov decision process, the probabilities given by p completely characterize the environment's dynamics.
- The probability of each possible value for S_t and R_t depends only on the immediately preceding state and action, S_{t-1} and A_{t-1}, and, given them, not at all on earlier states and actions
- The state must include information about all aspects of the past agent–environment interaction that make a difference for the future – Markov property



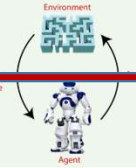
Value function

- The value function of a state s under a policy π, denoted $v_\pi(s)$, is the expected return when starting in s and following π thereafter.
- State-value function for policy π at time t is given as

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right], \text{ for all } s \in \mathcal{S}$$

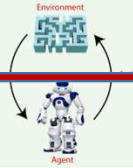
- The value of taking action a in state s under a policy π , denoted as $q_\pi(s, a)$, as the expected return starting from s, taking the action a, and thereafter following policy π

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a\right]$$



Value function – recursive relationship

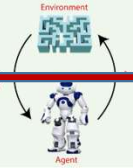
$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\
&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r \,|\, s, a) \Big[r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'] \Big] \\
&= \sum_a \pi(a|s) \sum_{s', r} p(s', r \,|\, s, a) \Big[r + \gamma v_\pi(s') \Big], \quad \text{for all } s \in \mathcal{S},
\end{aligned}
$$



Optimal values and optimal policies

- Optimal state value function $$v_*(s) \doteq \max_\pi v_\pi(s)$$
- Optimal action value function $$q_*(s,a) \doteq \max_\pi q_\pi(s,a)$$
- Optimal action value function as a function of optimal state value function

$$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$



Optimal values

$$
\begin{aligned}
v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s,a) \\
&= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\
&= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\
&= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_*(s')\big].
\end{aligned}
$$

$$
\begin{aligned}
q_*(s,a) &= \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a\Big] \\
&= \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma \max_{a'} q_*(s', a')\Big].
\end{aligned}
$$

